

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Bibliotecas: Pandas, NumPy, SciPy, Matplotlib e Seaborn

Numpy - manipulação de array

Aula 4

Código da aula: [DADOS]ANO1C2B3S22A4

Bibliotecas: Pandas,
NumPy, SciPy,
Matplotlib e Seaborn

Mapa da Unidade 5 Componente 3

semana

20

Numpy – acesso,
inspeção e índice

semana

21

Numpy – operação
de arrays

semana

22

Você está aqui!
Numpy –
manipulação de
array

Bibliotecas: Pandas,
NumPy, SciPy,
Matplotlib e Seaborn

Mapa da Unidade 5 Componente 3

Você está aqui!

Numpy - manipulação de array

Aula 4

Código da aula: [DADOS]ANO1C2B3S22A4

22



Objetivos da aula

- Conhecer as manipulações de arrays usando a biblioteca NumPy;
- Conhecer o MNIST.



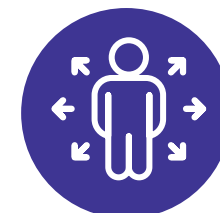
Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



Duração da aula

50 minutos.



Competências técnicas

- Proficiência em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Usar técnicas para explorar e analisar dados, aplicar modelos estatísticos, identificar padrões, realizar inferências e tomar decisões baseadas em evidências.



Competências socioemocionais

- Colaborar, efetivamente, com outros profissionais, como cientistas e engenheiros de dados;
- Trabalhar em equipes multifuncionais, colaborando com colegas, gestores e clientes

Construindo o conceito

Reshape – Numpy

Vamos explorar algumas combinações possíveis de matrizes com dimensões 128 x 128 usando o método reshape do NumPy:

1) Matriz 1D (vetor):

Você pode criar uma matriz unidimensional (vetor) com 16.384 elementos ($128 * 128$). Por exemplo:

```
import numpy as np
vetor = np.arange(16384)
vetor
```

```
array([ 0, 1, 2, ..., 16381, 16382, 16383])
```

```
vetor_reshaped = vetor.reshape(128, 128)
vetor_reshaped
```

```
array([[ 0, 1, 2, ..., 125, 126, 127],
       [128, 129, 130, ..., 253, 254, 255],
       [256, 257, 258, ..., 381, 382, 383],
       ...,
       [16000, 16001, 16002, ..., 16125, 16126, 16127],
       [16128, 16129, 16130, ..., 16253, 16254, 16255],
       [16256, 16257, 16258, ..., 16381, 16382, 16383]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Reshape – Numpy

2) Matriz 2D (imagem em escala de cinza):

Você pode representar uma imagem em escala de cinza com 128 x 128 pixels. Cada pixel pode ter um valor de intensidade (de 0 a 255).

Por exemplo:

```
imagem_cinza = np.random.randint(0, 256, size=(128, 128))  
imagem_cinza
```

```
array([[ 78,  63, 229, ...,  19, 146, 233],  
       [121, 214,  54, ..., 233, 141,  13],  
       [153,  75,  31, ...,  16,  79, 116],  
       ...,  
       [ 20,  81, 234, ...,  23,  93, 107],  
       [ 55, 147, 254, ..., 101,  52,  90],  
       [ 76,  97, 233, ..., 213, 234,  10]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Reshape – Numpy

3) Matriz 3D (imagem colorida):

Você pode criar uma imagem colorida com 128 x 128 pixels e 3 canais (RGB). Cada canal representa uma cor (vermelho, verde e azul).

Por exemplo:

```
imagem_colorida = np.random.randint(0, 256, size=(128, 128, 3))  
imagem_colorida
```

```
array([[[104, 125, 113],  
       [ 53,  51, 131],  
       [235, 227, 190],  
       ...,  
       [ 95,  95, 115],  
       [ 43, 186,  82],  
       [116, 247, 235]],  
      [[187, 171, 168],  
       [127, 191, 136],  
       [ 29, 171,   5],  
       ...,  
       [244,  31, 172],  
       [142, 235,  62],  
       [101, 251, 234]],  
      [[201, 162,  46],  
       [ 69,  73, 151],  
       [207,  65, 189],
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Reshape – Numpy

4) Tensor 3D (vídeo ou sequência de imagens):

Você pode criar um tensor 3D para representar uma sequência de imagens (como um vídeo). Cada imagem tem dimensões 128 x 128 e 3 canais (RGB).

Por exemplo:

```
video_frames = np.random.randint(0, 256, size=(10, 128, 128, 3))  
video_frames
```

```
array([[[[117, 108, 197],  
         [109,  10, 144],  
         [228, 134, 147],  
         ...,  
         [ 80,  26,  16],  
         [220, 124, 197],  
         [248, 137, 160]],  
       [[154, 131, 238],  
         [ 46,  88,  80],  
         [123, 127,  79],  
         ...,  
         [205,  35, 158],  
         [215,  81,  64],  
         [228, 170, 237]],  
       [[178, 186, 103],  
         [ 29, 104,  45],  
         [ 44,  63, 114],  
         ...,  
         [ 25, 161, 243],  
         [ 78, 160,  88],  
         [235, 129,   4]]],  
      dtype=uint8)
```

```
[[[R, G, B], [R, G, B], ...], # Primeira imagem  
 [[R, G, B], [R, G, B], ...], # Segunda imagem  
 ...]
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Colocando
em **prática**

Redimensionando imagens em escala de cinza

Analise:

Você está ajudando um cientista de dados e está trabalhando em um projeto de processamento de imagens. Sua tarefa é redimensionar imagens em escala de cinza para um tamanho específico. Aqui está o cenário:

Situação fictícia elaborada especialmente para o curso.

- ▶ Você recebeu uma imagem em escala de cinza, que é representada como uma matriz NumPy.
- ▶ A imagem tem 64 x 256.
- ▶ Seu objetivo é redimensionar a imagem para um tamanho padrão de 128 x 128.

Realize a função reshape de acordo com o cenário apresentado.



Durante a aula



Em grupo

Ser
sempre +

Situação

Imagine que você está trabalhando com processamento de imagens e recebeu uma imagem em formato de vetor linear (1D), que contém os valores em pixels. A imagem original é uma matriz 2D, mas foi achatada em um vetor 1D para transmissão ou armazenamento. Agora, você precisa reconstruir a imagem para seu formato original (2D) para processamento adicional.

Problema:

Você tem um vetor 1D (em pixels), que representa uma imagem em escala de cinza achatada, com um total de 48 unidades de pixel. Seu objetivo é remodelar esse vetor para uma matriz 2D de 6 x 8 para recuperar a imagem original.

Vetor 1D (em pixels):

[120, 125, 129, ..., 135, 130, 126]

(Os valores representam os níveis de cinza dos pixels, variando de 0 a 255.)

Matriz 2D após o reshape:

[[120, 125, 129, ..., 132],
 [128, 127, 130, ..., 131],

...

[135, 130, 126, ..., 129]]

(A matriz resultante representa a imagem em escala de cinza 6 x 8.)

Ser
sempre +

Situação

Para resolver esse problema, você usaria o método reshape do NumPy para converter o vetor 1D em uma matriz 2D.

Aqui está um exemplo de código que faz isso:

```
import numpy as np

# Vetor 1D com valores de pixels (exemplo simplificado)
pixels = np.array([120, 125, 129, ..., 135, 130, 126])

# Reshape para uma matriz 2D de 6x8
imagem_2d = pixels.reshape((6, 8))

# Verifique se o reshape foi bem-sucedido
print(imagem_2d)
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

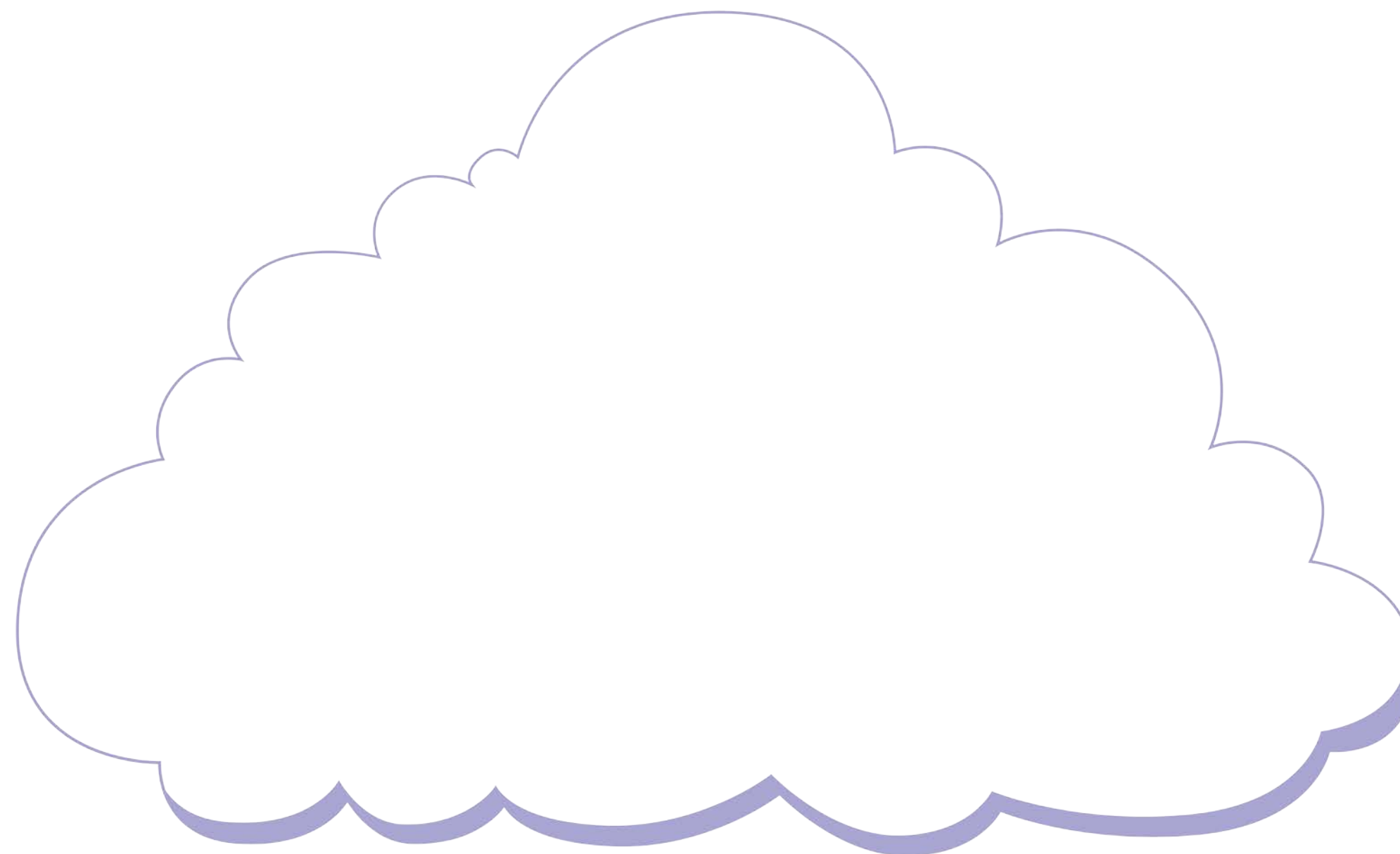
Ser
sempre +

Ação

Esse tipo de operação é comum em aplicações de visão computacional e aprendizado de máquina, nas quais a manipulação da forma dos dados é necessária para diferentes etapas de processamento e análise. Então, diversos cenários podem acontecer.

- ▶ Na sua percepção, que erros podem acontecer nesse cenário?

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**

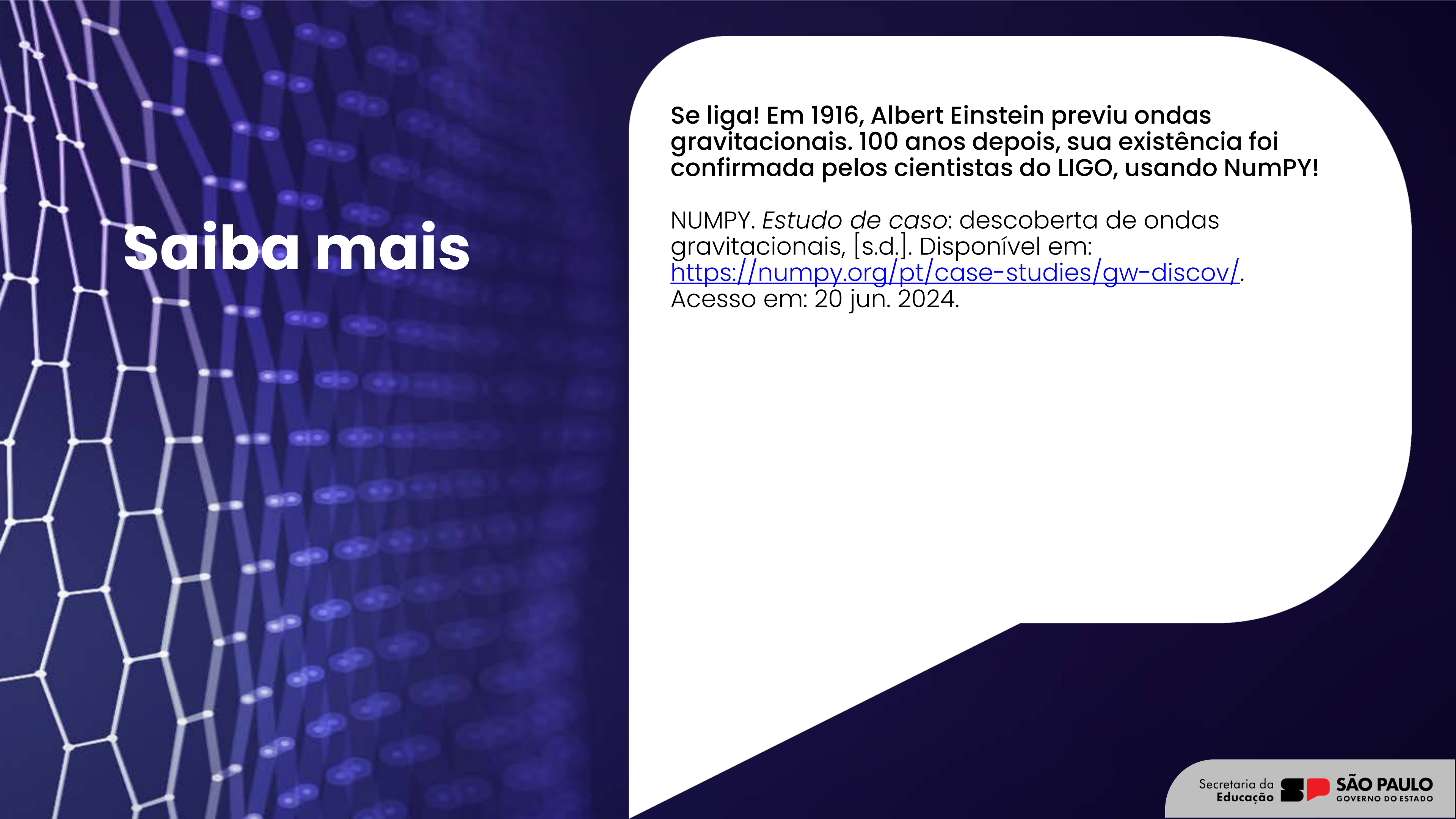


© Getty Images

O que nós
**aprendemos
hoje?**

Então, ficamos assim...

- 1** Existem várias dimensões do NumPy, assim como de reshape.
- 2** Pode-se utilizar o -1 para o NumPy inferir, automaticamente, o valor da dimensão do reshape.
- 3** Para o redimensionamento do NumPy usa-se o reshape e deve-se manter a quantidade de elementos.



Saiba mais

Se liga! Em 1916, Albert Einstein previu ondas gravitacionais. 100 anos depois, sua existência foi confirmada pelos cientistas do LIGO, usando NumPY!

NUMPY. *Estudo de caso: descoberta de ondas gravitacionais*, [s.d.]. Disponível em: <https://numpy.org/pt/case-studies/gw-discov/>. Acesso em: 20 jun. 2024.

Referências da aula

MCKINNEY, W. *Python para análise de dados: tratamento de dados com Pandas, NumPy & Jupyter*. São Paulo: Novatec, 2023.

NUMPY. *Página inicial*: versão 1.26.0, 16 set. 2023. Disponível em: <https://numpy.org/pt/>
Acesso em: 20 jun. 2024.

Identidade visual: Imagens © Getty Images

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**